Emandoyesus Tesfay
ugr/188057/16

Architectural Significant Requirement (ASR) by perspective

Movie Explorer System

1. Frontend Architecture

- Built as a Single Page Application (SPA) using React.js for a fast and dynamic UX.
- Uses a component-based structure (navbar, search, filters, movie cards) for modularity.
- State management with React hooks (useState) for handling authentication, search, and watchlists.
- Performance optimization through lazy loading, code splitting, and memoization.
- Ensures security and responsiveness using input validation, XSS protection, HTTP-only cookies, and Tailwind CSS.

2. Backend Architecture

- Uses Node.js with Express.js for scalable and efficient request handling.
- Follows a layered architecture (controllers, services, data access) for maintainability.
- Handles authentication, business logic, and API integration (e.g., TMDB).
- Uses JWT authentication for secure and stateless communication.
- Optimized with asynchronous processing and security measures (rate limiting, HTTPS).

3. DevOps Architecture

- Git and GitHub will be used for version control and collaboration.
- Implements CI/CD with GitHub Actions for automated testing and deployment.
- Relies on platform-managed deployments (e.g., Vercel) to reduce complexity.
- Supports monitoring and debugging via logs and platform analytics tools.
- Ensures stability with rollback mechanisms and error monitoring.

4. Database Architecture

- **Neon** (PostgreSQL cloud service) will be used for structured, scalable, and reliable data storage.
- Schema will be designed for users, watchlists, favorites, and cached movie metadata.
- Indexing will be applied to optimize queries for fast retrieval of movie and user data.
- Backup and recovery strategies will be implemented to prevent data loss.
- Security will be enforced with access control and encryption for sensitive information.

5. Deployment Environment

- Frontend will be deployed on **Vercel** for fast global delivery and automatic builds.
- Backend will be hosted on **Render** for continuous deployment and server management.
- Database will be managed with **Neon** (PostgreSQL cloud service).
- Uses secure HTTPS communication between all system components.
- Implements automated deployment pipeline via GitHub integration.